MESCLA INVEST

Relatório da parte de autenticação

Documento explicando, ponto a ponto, o que foi implementado no fluxo de autenticação do projeto: login, cadastro, primeiro acesso, 2FA, recuperação de senha e desbloqueio biométrico.

Objetivo do relatório
Mostrar o caminho completo que o app percorre desde a abertura até a entrada no dashboard, destacando a lógica que você integrou no Flutter, nas Services e nas Cloud Functions.

BASE USADA
Estado atual do repositório local.

FOCO
Autenticação, MFA, reset de senha e biometria.

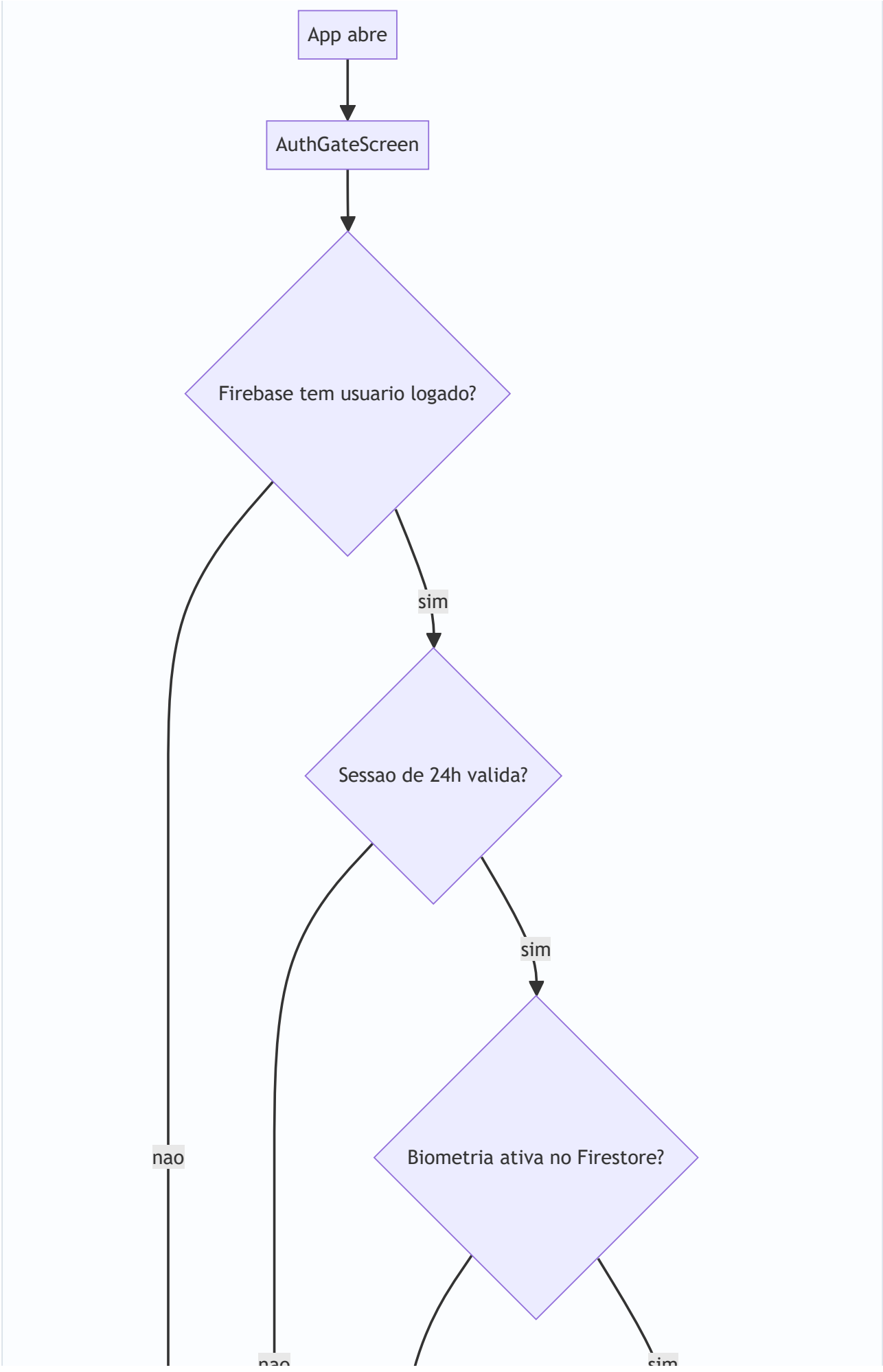
ARQUIVOS PRINCIPAIS
`lib/auth/*` e `functions/src/auth/*`.

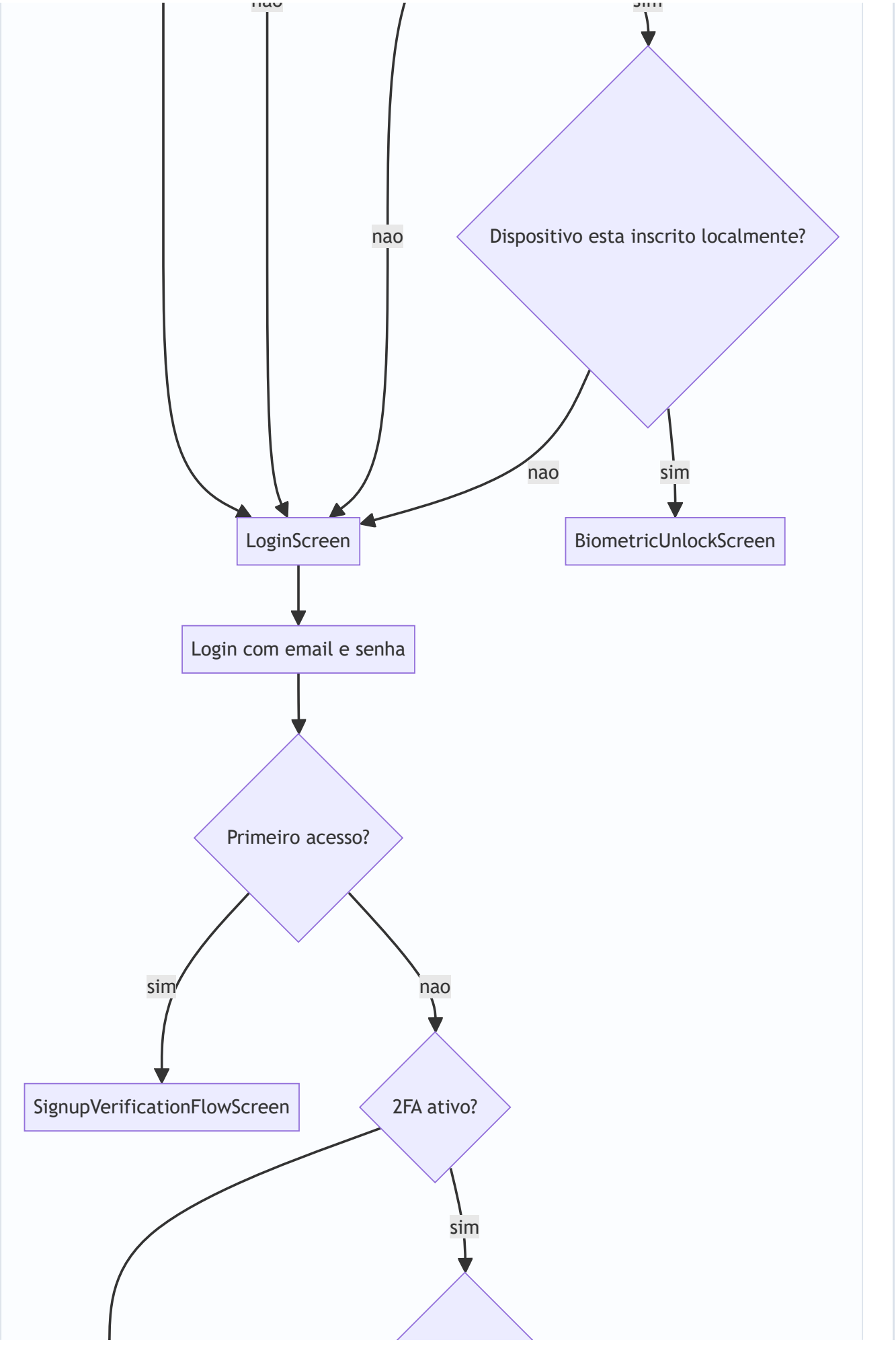
RESULTADO
Fluxo integrado do login inicial até o acesso final ao dashboard.

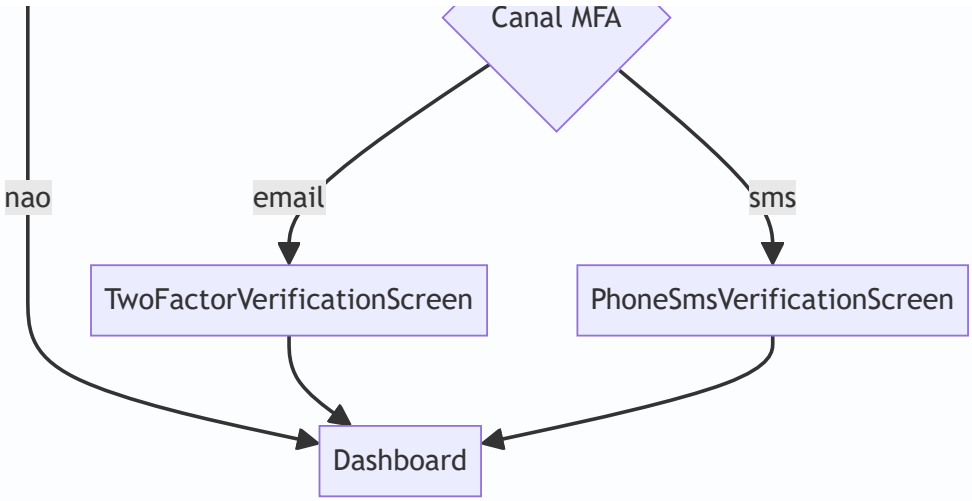
O relatório foi escrito para ficar legível em PDF e também para servir como explicação oral do que você fez.

1. Visão geral do fluxo

A autenticação foi organizada como uma cadeia de decisões. O app não vai direto para a tela principal: primeiro ele verifica se existe uma sessão válida, se a biometria está liberada, se o usuário está no primeiro acesso e se o 2FA precisa ser aplicado.


App abre
AuthGateScreen
Firebase tem usuario logado?
sim
nao
Sessao de 24h valida?
sim
Biometria ativa no Firestore?


nao
sim
nao
Dispositivo esta inscrito localmente?
nao
sim
LoginScreen
BiometricUnlockScreen
Login com email e senha
Primeiro acesso?
sim
nao
SignupVerificationFlowScreen
2FA ativo?
sim



O que esse desenho mostra

- O app trata autenticação como uma sequência de filtros, não como uma única chamada.
- Cada condição protege a seguinte: sessão, biometria, primeiro acesso, 2FA.
- Isso reduz navegação errada e centraliza a decisão em poucos pontos do código.

Por que isso é importante

- O usuário não entra no dashboard sem passar pelas regras certas.
- O app consegue retomar estado sem depender de telas soltas.
- Os dados do Firestore funcionam como fonte de verdade para preferências de segurança.

2. Porta de entrada: `AuthGateScreen`

Essa foi a tela que você colocou logo após o splash. Ela funciona como porteiro do sistema: decide se o usuário pode usar o atalho biométrico ou se precisa cair no login normal.

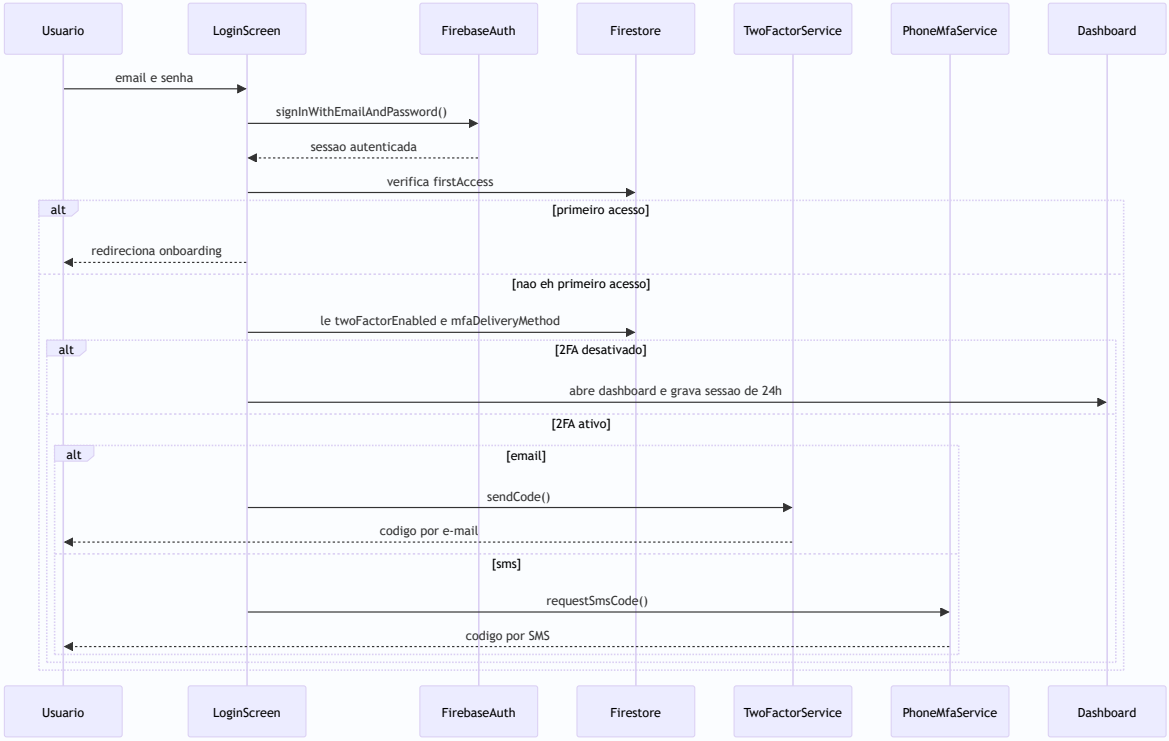
1. Ao iniciar, `initState()` dispara `_bootstrap()` para fazer a checagem assíncrona sem travar a UI.
2. O código verifica se o Firebase já foi inicializado e se existe um `currentUser`.
3. Depois lê a validade da sessão local de 24h em `SessionPersistenceService`.
4. Em seguida consulta `biometricEnabled` no Firestore e o enrollment local do aparelho.
5. Se tudo bater, o app abre `BiometricUnlockScreen`.
6. Se qualquer peça falhar, o fluxo faz `signOut()` e volta para `LoginScreen`.

> **Decisão técnica importante**
> Você não deixou a autenticação “meio aberta”. Quando o atalho biométrico não é possível, o estado é limpo e o app volta para o login. Isso evita sessão antiga, aparelho trocado ou enrolamento local inconsistente.

3. Login com email, senha e 2FA

A `LoginScreen` ficou responsável por autenticar, validar primeiro acesso e decidir qual caminho seguir depois do login. Ela também passou a controlar o envio do 2FA por e-mail ou SMS.

1. O usuário informa e-mail e senha.
2. O app faz uma validação local simples para não chamar o Firebase com campos vazios.
3. O login real é feito via `UserFirestoreService.signInWithEmailAndPassword()`.
4. Depois da autenticação, o fluxo verifica se o usuário ainda está no primeiro acesso.
5. Se for primeiro acesso, o app abre a etapa de onboarding/validação.
6. Se não for, lê a flag `twoFactorEnabled` e decide se o usuário entra direto ou não.
7. Quando 2FA está ativo, o campo `mfaDeliveryMethod` define se o código vai por e-mail ou por SMS.



O que você resolveu nesse ponto

- Separou autenticação de navegação, para o login não virar um bloco confuso.
- Tratou erros do Firebase com mensagens legíveis para o usuário.
- Impediu navegação errada com checagem de `mounted` após chamadas assíncronas.
- Passou a gravar a sessão local só quando o login realmente foi concluído.

4. Cadastro em múltiplas etapas

O cadastro foi desenhado como uma jornada completa, e não como um formulário único. Isso melhora a experiência e permite validar cada informação separadamente.

As etapas que você implementou cobrem: nome, e-mail, telefone, escolha do MFA, CPF, senha, confirmação de senha, aceite de termos e foto de perfil.

Etapa	Validação	Motivo técnico
Nome	Nome e sobrenome	Garante perfil mais completo e avatar com iniciais corretas.
E-mail	Formato plausível	Evita erros logo no início do fluxo.
Telefone	Celular BR com DDD e 9 dígitos	Necessário para SMS/MFA e link de telefone.
MFA	Email ou SMS	Define a estratégia de segundo fator.
CPF	Validador de dígitos verificadores	Evita documento inválido persistido no cadastro.
Senha	8+ caracteres, maiúscula, número, símbolo	Aumenta a segurança mínima da conta.
Termos	Leitura até o fim + aceite	Garante consentimento explícito.
Foto	Opcional	Melhora o perfil sem bloquear o cadastro.

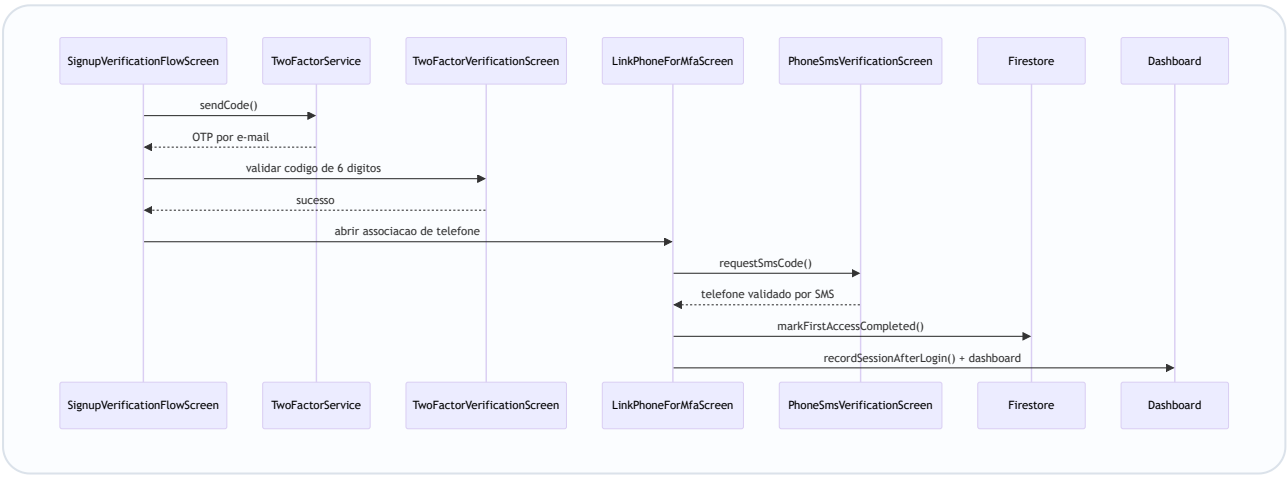
No fim do fluxo, o app chama `UserFirestoreService.createUserWithEmailAndPassword()`. Essa função cria o usuário no Firebase Auth e grava o documento em `users/{uid}` no Firestore. Os defaults já nascem prontos: `twoFactorEnabled: true`, `firstAccess: true`, `biometricEnabled: false`, lista vazia de favoritos, lista vazia de chaves PIX e o canal MFA escolhido pelo usuário.

> **Depois do cadastro**
> O app não finaliza ali. Ele abre `SignupVerificationFlowScreen`, que conclui a validação inicial antes de liberar o dashboard.

5. Primeiro acesso: e-mail, telefone e liberação do dashboard

O fluxo de primeiro acesso existe para amarrar identidade real do usuário com a conta recém-criada. Ele confirma o e-mail via código, depois confirma o telefone via SMS, e só então marca `firstAccess = false`.

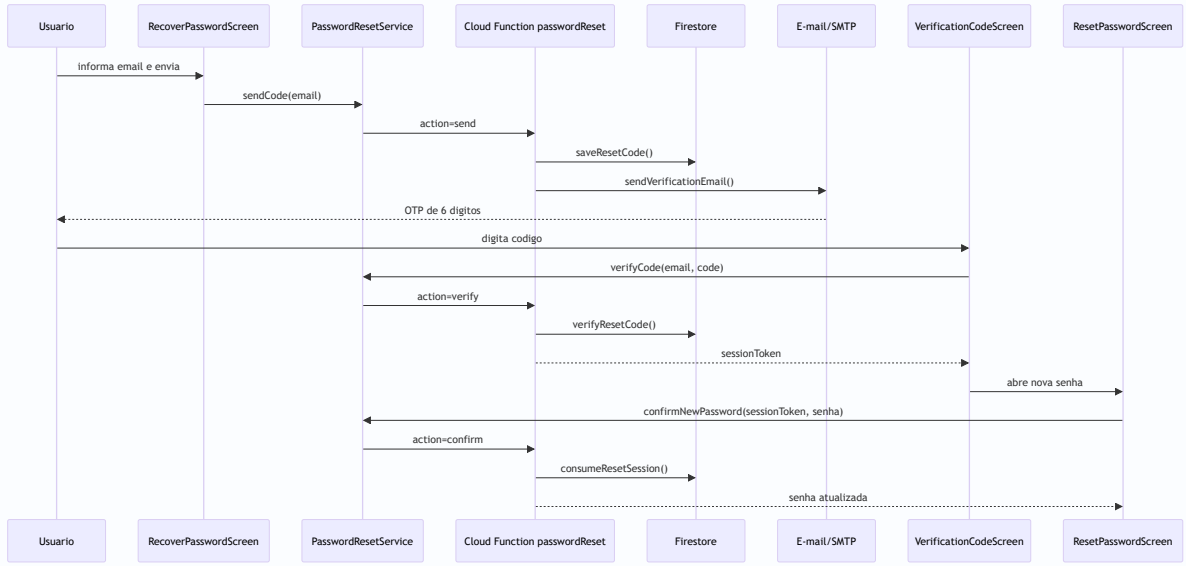
1. `SignupVerificationFlowScreen` envia o primeiro OTP por e-mail usando `TwoFactorService.sendCode()`.
2. O usuário valida o código na `TwoFactorVerificationScreen`.
3. Depois disso, a tela abre `LinkPhoneForMfaScreen`.
4. O número é pré-preenchido com o telefone salvo no Firestore.
5. O SMS é enviado e validado em `PhoneSmsVerificationScreen`.
6. Se tudo der certo, o sistema chama `markFirstAccessCompleted()`.
7. Por fim, grava a sessão quente de 24h e abre o dashboard.



Esse trecho é importante porque mostra que você não tratou o cadastro como uma simples criação de conta. Você transformou o cadastro em uma etapa de verificação e preparo do perfil para uso real.

6. Recuperação de senha com OTP

A recuperação de senha foi feita em três etapas e não depende do reset padrão do Firebase Auth. Isso foi necessário porque a interface do projeto trabalha com OTP de 6 dígitos, não com link de recuperação por `oobCode`.



O papel do app

- Validar e-mail localmente.
- Chamar a callable `passwordReset`.
- Receber e repassar o `sessionToken` com segurança.
- Mostrar mensagens claras em caso de erro.

O papel do servidor

- Gerar OTP.
- Salvar só hash no Firestore.
- Controlar tentativas e expiração.
- Autorizar a troca final de senha via Admin SDK.

O ponto mais importante aqui é o `sessionToken`: ele funciona como prova temporária de que o usuário validou o e-mail. Sem isso, a etapa final não consegue trocar a senha.

7. 2FA no login e SMS por telefone

Quando a conta pede segundo fator, você fez o app escolher entre e-mail e SMS com base no campo `mfaDeliveryMethod`. Assim o fluxo ficou flexível e preparado para o cenário real do usuário.

2FA por e-mail

- `LoginScreen` chama `TwoFactorService.sendCode()`.
- A Cloud Function `twoFactor` gera e grava o OTP no Firestore.
- `TwoFactorVerificationScreen` valida os 6 dígitos.
- Se acertar, o app grava a sessão de 24h e segue para o dashboard.

2FA por SMS

- `PhoneMfaService` usa o Firebase Phone Auth.
- Se não houver telefone associado, o app pede a vinculação antes de continuar.
- A verificação por SMS serve tanto para login quanto para associação do telefone.
- O mesmo componente de 6 dígitos é reaproveitado nas duas situações.

Esse desenho evita duplicação. A UI para OTP por e-mail e por SMS é parecida, mas o backend e a origem do código são diferentes. Você isolou isso em serviços próprios, sem misturar com a tela.

8. Camada de dados, segurança e erros

A camada `UserFirestoreService` foi o centro de persistência do fluxo. Ela concentra leitura e escrita de flags como `firstAccess`, `twoFactorEnabled`, `mfaDeliveryMethod`, `biometricEnabled` e dados de perfil do usuário.

- **Primeiro acesso:** `firstAccess` começa como `true` e só cai para `false` ao final da validação inicial.
- **2FA:** o campo `twoFactorEnabled` controla se o login exige OTP.
- **Canal MFA:** `mfaDeliveryMethod` escolhe entre e-mail e SMS.
- **Biometria:** `biometricEnabled` e o enrollment local permitem o atalho de desbloqueio.
- **Sessão quente:** `SessionPersistenceService` registra 24h após login bem sucedido.
- **Erro legível:** `AuthService` traduz exceções do Firebase para mensagens humanas.

Decisões de segurança que ficaram bem resolvidas

- OTP armazenado em hash, nunca em texto puro.
- Links e códigos com expiração curta.
- Respostas neutras quando a função não deve enumerar contas.
- Limite de tentativas para impedir abuso.
- Limpeza da sessão ao sair da conta ou ao invalidar o atalho biométrico.
- Uso de `mounted` após `await` para evitar navegação com widget desmontado.

Em resumo, a autenticação não ficou só “bonita na tela”. Ela passou a ter regras consistentes, estados persistidos, tratamento de falhas e uma cadeia clara de autorização até o dashboard.

Arquivos centrais do fluxo: `lib/auth/screens/auth_gate_screen.dart`, `login_screen.dart`, `create_account_screen.dart`, `signup_verification_flow_screen.dart`, `recover_password_screen.dart`, `verification_code_screen.dart`, `reset_password_screen.dart`, `two_factor_verification_screen.dart`, `link_phone_for_mfa_screen.dart`, `phone_sms_verification_screen.dart`, `user_firestore_service.dart`, `password_reset_service.dart`, `two_factor_service.dart` e `functions/src/auth/*`.